Galton Board Simulator

History, probability theory, statistical interpretation, software architecture, installation, operation, validation, and packaging

The **Galton Board Simulator** is an animated educational and scientific visualisation of a Galton board, also called a **quincunx**, **bean machine**, or **probability machine**. A sequence of binary left/right decisions sends each ball through a triangular peg lattice into one of the receiving bins. Although the trajectory of one ball is unpredictable, the aggregate distribution of many balls is highly structured and is described exactly by the binomial distribution.

This repository combines three distinct objectives:

1. **Historical interpretation** - explain the origin of the Galton board and its role in the development of statistics.
2. **Mathematical interpretation** - derive the Bernoulli, binomial, random-walk, Pascal-triangle, law-of-large-numbers, and central-limit-theorem results demonstrated by the board.
3. **Software interpretation** - document the Python/Tkinter simulator, its controls, architecture, numerical assumptions, limitations, installation, and Windows executable packaging.

The simulator uses an **exact probability model** rather than an approximate rigid-body collision solver. Each row performs one independent Bernoulli trial. The animation then displays a smooth, realistic-looking trajectory consistent with the already-selected sequence of left/right outcomes. Fall speed, bounce amplitude, trails, ball gloss, collision rings, and spark particles are visual parameters; they do not change the selected probability law.

Quick start

Requirements

- Python 3.10 or newer is recommended.
- Tkinter/Tk 8.6 is required.
- No third-party package is required to run the simulator.
- PyInstaller is optional and is used only to create a Windows executable.

Run directly

Open Command Prompt in the repository directory and execute:

```
python script.py
```

The program opens maximised on Windows.

Basic controls

Action	Control
Pause or resume	`Space`
Reset the experiment	`R`

Action	Control
Release one ball	`N`
Release 100 balls	`B`
Increase release rate	`Up Arrow`
Decrease release rate	`Down Arrow`
Show or hide help	`H`
Exit	`Esc`

What the simulator demonstrates

The simulator illustrates a central theme of probability and statistical mechanics:

Microscopic unpredictability can coexist with macroscopic regularity.

A single ball follows a random path. Before the ball is released, its final bin is unknown. Nevertheless, if many independent balls are released under the same conditions, the fraction entering each bin approaches a deterministic probability.

For a board with n rows and a probability p of moving right at each row, the final bin index K has the distribution

$$K \sim \text{Binomial}(n, p).$$

The probability of finishing in bin k is

$$P(K = k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad k = 0, 1, \ldots, n.$$

For the symmetric case $p = 1/2$,

$$P(K = k) = \frac{\binom{n}{k}}{2^n}.$$

The distribution is symmetric around $n/2$. When n is sufficiently large, its shape is well approximated by a normal distribution.

The simulator also displays:

- the current bin counts;
- the theoretical binomial expectation;
- Pascal-triangle coefficients;
- expected percentages;
- the theoretical mean and standard deviation;
- the observed mean and standard deviation;
- a normal-approximation formula;
- a moving-ball count and frame-rate estimate.

What a Galton board is

A classical Galton board consists of:

- a vertical or inclined backing board;
- a release funnel at the top;
- staggered rows of pegs;
- small balls, beads, pellets, or grains;
- vertical partitions forming receiving bins at the bottom;
- frequently, a transparent front cover to keep the balls in the plane of the apparatus.

Each time a ball encounters a peg, small differences in contact position, velocity, surface roughness, spin, alignment, and vibration determine whether the ball leaves the peg on the left or the right. In an ideal symmetric probability model, left and right are assigned equal probability.

A board with n peg rows has $n+1$ possible receiving bins because the ball may move right zero times, one time, two times, and so on up to n times.

The apparatus is known by several names:

- **Galton board** - the most common statistical name;
- **quincunx** - Galton's historical term and a reference to staggered point patterns;
- **bean machine** - a common educational name;
- **probability machine** - emphasises its mathematical purpose;
- **Plinko-like board** - an informal comparison with the television game, although game-show boards may differ in geometry, boundary conditions, prize layout, and physical bias.

Historical development

Probability before Galton

The mathematical ideas demonstrated by the Galton board predate the physical apparatus.

Pascal and combinatorial counting

Blaise Pascal's seventeenth-century work on arithmetic triangles organised the coefficients now written as

$$\binom{n}{k}.$$

The same coefficients count:

- subsets of size k selected from n objects;
- sequences containing k successes and $n-k$ failures;
- lattice paths containing k right moves and $n-k$ left moves;
- the coefficients of the binomial expansion $(a+b)^n$.

The arithmetic triangle was known in several mathematical traditions before Pascal, but his work helped establish its role in European probability and combinatorics.

Bernoulli and repeated trials

Jacob Bernoulli's *Ars Conjectandi*, published posthumously in 1713, developed the mathematics of repeated independent trials and contained an early form of the law of large numbers. The Bernoulli-trial model is the direct abstract ancestor of the ideal Galton board.

De Moivre and the normal approximation

Abraham de Moivre investigated repeated coin tossing and showed that the central part of the binomial distribution could be approximated by a bell-shaped exponential curve. His work is an early form of the normal approximation to the binomial distribution.

Laplace and the central limit tradition

Pierre-Simon Laplace greatly generalised the approximation of sums of random variables. The de Moivre-Laplace theorem is now understood as a special case of the central limit theorem.

Quetelet and the "average man"

Adolphe Quetelet applied probability distributions and the law of errors to human measurements and social data. His idea of the *average man* influenced nineteenth-century attempts to describe biological and social variation statistically. Galton's study of heredity developed in this intellectual setting.

Francis Galton

Francis Galton (1822-1911) was a British polymath and a cousin of Charles Darwin. His work included geography, meteorology, anthropometry, fingerprints, heredity, correlation, regression, and statistical graphics.

Galton delivered the Royal Institution lecture **"Typical Laws of Heredity"** on 9 February 1877. The Galton board, or quincunx, was used as a visual model of accumulated variation. An individual ball appeared to take an irregular path, while many balls formed a stable bell-shaped aggregate.

In the hereditary interpretation, each deflection represented a small influence on a biological trait. Galton compared the distribution of balls with distributions of measured human characteristics, especially stature.

Descriptions and developments of the apparatus also appeared in Galton's later work, notably *Natural Inheritance* (1889).

The two-stage quincunx and regression toward the mean

Galton noticed a conceptual difficulty. If each generation merely added a new independent set of deviations, variation would continually increase:

$$\operatorname{Var}(X_1 + \cdots + X_n) = \sum_{i=1}^{n} \operatorname{Var}(X_i)$$

for independent increments.

Human-height distributions do not widen without limit from generation to generation. Galton therefore demonstrated a two-stage quincunx with channels that contracted the first-stage distribution toward the centre before a second set of random deviations was added.

This was Galton's mechanical analogy for what he initially called **reversion** and later **regression toward mediocrity**, now called **regression toward the mean**. Modern statistics treats regression toward the mean as a general consequence of imperfect correlation and selection on extreme observations, not as a universal physical restoring force.

Correlation and regression

Galton's investigations contributed to the later formal development of:

- correlation;
- linear regression;
- regression toward the mean;
- bivariate distributions;
- quantitative analysis of heredity.

Karl Pearson subsequently formalised and extended much of this statistical programme.

Modern mathematical developments

The Galton board has become more than a classroom illustration.

Mechanical and dynamical simulations

Kozlov and Mitrofanova studied a mechanical model with gravity, circular nails, and a coefficient of restitution. Their simulations showed that the resulting distribution depends on impact elasticity, peg geometry, and the initial release distribution. A real board is therefore not automatically an exact binomial machine.

Galton board as a driven Lorentz gas

In mathematical physics, an idealised infinite Galton board can be formulated as a periodic Lorentz gas under an external field. Chernov and Dolgopyat studied limit laws, anomalous scaling, and recurrence in this system. Their model is considerably more complex than the finite Bernoulli board implemented here.

Hilbert-Galton board

Ayyer and Ramassamy introduced a Markov-chain variant in which balls arrive in bins at specified rates and the entire bin configuration can shift. The resulting Hilbert-Galton board connects the classical device to stationary distributions, triangular arrays, spectra, coupling, and interacting-particle systems.

Generalised boards and statistical physics

Recent work has used Galton-board-style microscopic update laws to explain how repeated elementary interactions can produce Gaussian, lognormal, Pareto, beta, and other macroscopic distributions. These extensions connect the board with kinetic theory, economics, opinion dynamics, biological populations, and social modelling.

Historical and ethical context

A technically complete history must distinguish Galton's mathematical contributions from the social programme with which some of his work was associated.

Galton coined the term **eugenics** and advocated selective human breeding. Eugenics later contributed to coercive policies, forced sterilisation, exclusion, racial hierarchy, and other serious abuses. Those ideas are scientifically and ethically unacceptable.

The Galton board remains valuable as a demonstration of probability, combinatorics, random walks, and statistical convergence. Its use does not require acceptance of Galton's social conclusions. Responsible teaching should:

- acknowledge the historical connection rather than omit it;
- separate valid mathematical ideas from invalid social and biological claims;
- avoid treating statistical regularity as a justification for ranking people;
- distinguish population-level distributions from deterministic claims about individuals;
- recognise that measured human traits are shaped by biological, environmental, social, and measurement processes;
- explain that statistical models are descriptions under assumptions, not moral prescriptions.

From a physical board to a probability model

A real falling ball is governed by mechanics. An ideal probability board is governed by a stochastic abstraction.

These are related but not identical descriptions.

Physical description

A physical trajectory depends on:

- gravitational acceleration;
- initial position and velocity;
- ball radius and mass;
- peg radius and spacing;
- board inclination;
- coefficient of restitution;
- rolling and sliding friction;
- spin;
- air resistance;
- peg and ball deformation;
- simultaneous or near-simultaneous contacts;
- vibrations and manufacturing tolerances;
- side-wall interactions.

In principle, deterministic mechanics may determine the trajectory if the complete initial state and all contact laws are known exactly. In practice, tiny unmeasured differences are amplified into different left/right outcomes.

Stochastic abstraction

The ideal model replaces the inaccessible contact details with one binary random variable per row:

$$X_i = \begin{cases} 1, & \text{right}, \\ 0, & \text{left}. \end{cases}$$

For a symmetric board,

$$P(X_i = 1) = P(X_i = 0) = \frac{1}{2}.$$

The abstraction discards detailed impact mechanics but preserves the combinatorial feature responsible for the receiving-bin distribution: the total number of right moves.

This is an example of model reduction. A complicated microscopic process is replaced by a simpler stochastic rule that retains the observable of interest.

Foundations of probability

Sample space

A **sample space** Ω is the set of all possible elementary outcomes.

For one binary peg decision,

$$\Omega_1 = \{L, R\}.$$

For n rows,

$$\Omega_n = \{L, R\}^n.$$

The symmetric board has 2^n possible left/right sequences.

For $n = 3$,

$$\Omega_3 = \{LLL, LLR, LRL, LRR, RLL, RLR, RRL, RRR\}.$$

Events

An **event** is a subset of the sample space.

Examples:

- the ball finishes in the central bin;
- the ball moves right exactly k times;
- the first move is left;
- at least half the moves are right;
- the final displacement is positive.

Probability axioms

A probability measure P satisfies:

$$P(A) \geq 0$$

for every event A,

$$P(\Omega) = 1,$$

and, for pairwise disjoint events $A_1, A_2, \ldots$,

$$P\left(\bigcup_i A_i\right) = \sum_i P(A_i).$$

It follows that

$$P(A^c) = 1 - P(A)$$

and

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

Conditional probability

For $P(B) > 0$,

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Independence

Events A and B are independent when

$$P(A \cap B) = P(A)P(B).$$

For independent row decisions,

$$P(X_1 = x_1, \ldots, X_n = x_n) = \prod_{i=1}^{n} P(X_i = x_i).$$

Independence is an assumption of the ideal board. A physical board may violate it if one collision changes spin or horizontal velocity in a way that influences later collisions.

Random variables

A random variable maps outcomes to numerical values.

For a single row,

$$X_i : \Omega_n \to \{0, 1\}.$$

The final bin index is

$$K = \sum_{i=1}^{n} X_i.$$

Bernoulli trials

A **Bernoulli trial** has two possible outcomes.

Let

$$X \sim \text{Bernoulli}(p).$$

Then

$$P(X = 1) = p$$

and

$$P(X = 0) = 1 - p = q.$$

The probability mass function is

$$P(X = x) = p^x (1 - p)^{1-x}, \qquad x \in \{0, 1\}.$$

The mean is

$$\text{E}[X] = p.$$

The variance is

$$\text{Var}(X) = p(1 - p) = pq.$$

The standard deviation is

$$\sigma_X = \sqrt{pq}.$$

For $p = 1/2$,

$$\mathrm{E}[X] = \frac{1}{2}, \qquad \mathrm{Var}(X) = \frac{1}{4}.$$

The Galton board as a random walk

There are two common coordinate conventions.

Right-move-count convention

The simulator uses

$$K = \sum_{i=1}^{n} X_i,$$

where K is the receiving-bin index. Bin 0 corresponds to all left moves; bin n corresponds to all right moves.

Signed-displacement convention

Define

$$Y_i = \begin{cases} +1, & \text{right}, \\ -1, & \text{left}. \end{cases}$$

The signed random-walk displacement is

$$S_n = \sum_{i=1}^{n} Y_i.$$

Since $Y_i = 2X_i - 1$,

$$S_n = 2K - n.$$

Therefore,

$$K = \frac{S_n + n}{2}.$$

Only positions with the same parity as n are reachable in the signed coordinate. For example, after an even number of rows, S_n is even.

For general right probability p,

$$\mathrm{E}[S_n] = n(2p-1)$$

and

$$\mathrm{Var}(S_n) = 4np(1-p).$$

For $p = 1/2$,

$$\mathrm{E}[S_n] = 0$$

and

$$\mathrm{Var}(S_n) = n.$$

Thus the typical random-walk displacement grows like $\sqrt{n}$ rather than n.

The binomial distribution

If $X_1, \ldots, X_n$ are independent Bernoulli variables with the same success probability p, then

$$K = \sum_{i=1}^{n} X_i$$

has a binomial distribution:

$$K \sim \mathrm{Binomial}(n, p).$$

Derivation of the probability mass function

To finish in bin k, a path must contain:

- exactly k right moves;
- exactly $n-k$ left moves.

Any particular sequence with k right moves has probability

$$p^k(1-p)^{n-k}.$$

The number of different sequences containing k right moves is

$$\binom{n}{k}.$$

Therefore,

$$P(K = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

The binomial coefficient is

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

The support is

$$k = 0, 1, \ldots, n.$$

Normalisation

The probabilities sum to one:

$$\sum_{k=0}^{n} \binom{n}{k} p^k (1-p)^{n-k} = 1.$$

This follows directly from the binomial theorem:

$$(p + (1-p))^n = 1^n = 1.$$

Symmetric board

For $p = 1/2$,

$$P(K = k) = \frac{\binom{n}{k}}{2^n}.$$

Since

$$\binom{n}{k} = \binom{n}{n-k},$$

the distribution is symmetric:

$$P(K = k) = P(K = n - k).$$

Ratio of adjacent probabilities

The ratio

$$\frac{P(K = k+1)}{P(K = k)} = \frac{n-k}{k+1} \frac{p}{1-p}$$

is useful for stable recursive calculation and for locating the mode.

A binomial mode is

$$m = \lfloor (n+1)p \rfloor ,$$

with two adjacent modes when $(n+1)p$ is an integer.

Pascal's triangle and the binomial theorem

Pascal recurrence

The binomial coefficients satisfy

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

This recurrence has a direct path interpretation. A path entering bin k after n rows must arrive from:

- bin $k-1$ and move right; or
- bin k and move left.

For a symmetric board, the unnormalised path counts evolve exactly as Pascal's triangle.

First rows

```
n = 0                          1
n = 1                        1   1
n = 2                      1   2   1
n = 3                    1   3   3   1
n = 4                  1   4   6   4   1
n = 5                1   5  10  10   5   1
n = 6            1   6  15  20  15   6   1
```

Row sum

The sum of row n is

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

This is the number of binary sequences of length n.

Binomial theorem

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k.$$

Set $a = 1-p$ and $b = p$:

$$((1-p)+p)^n = \sum_{k=0}^{n} \binom{n}{k} (1-p)^{n-k} p^k = 1.$$

The binomial theorem therefore provides both the algebraic and probabilistic normalisation of the Galton-board distribution.

Mean, variance, standard deviation, skewness, and kurtosis

Let

$$K \sim \text{Binomial}(n, p)$$

and define

$$q = 1 - p.$$

Mean

$$\mu = \text{E}[K] = np.$$

The expected bin is the number of rows multiplied by the right probability.

Variance

$$\sigma^2 = \text{Var}(K) = npq.$$

Standard deviation

$$\sigma = \sqrt{npq}.$$

Interpretation

- Increasing n increases the absolute width of the distribution.
- The standard deviation grows as $\sqrt{n}$.
- The relative width σ/n decreases as $1/\sqrt{n}$.
- The variance is largest at $p = 1/2$.
- As p approaches 0 or 1, the distribution becomes narrower and more concentrated near an edge.

Skewness

The skewness is

$$\gamma_1 = \frac{1-2p}{\sqrt{npq}}.$$

Thus:

- $p = 1/2$ gives zero skewness;
- $p < 1/2$ gives positive skewness;
- $p > 1/2$ gives negative skewness;
- skewness decreases in magnitude as n grows.

Excess kurtosis

The excess kurtosis is

$$\gamma_2 = \frac{1 - 6pq}{npq}.$$

As n increases, γ_2 approaches zero, consistent with convergence toward the normal shape.

Generating functions and moments

Generating functions provide compact descriptions of a distribution.

Probability-generating function

For $K \sim \text{Binomial}(n, p)$,

$$G_K(z) = \mathrm{E}[z^K] = (1 - p + pz)^n.$$

The probability $P(K = k)$ is the coefficient of z^k.

Moment-generating function

$$M_K(t) = \mathrm{E}[e^{tK}] = (1 - p + pe^t)^n.$$

Differentiating at $t = 0$ gives moments:

$$\mathrm{E}[K] = M_K'(0) = np.$$

Also,

$$\mathrm{E}[K(K-1)] = M_K''(0) = n(n-1)p^2.$$

Then

$$\mathrm{E}[K^2] = n(n-1)p^2 + np,$$

and

$$\mathrm{Var}(K) = \mathrm{E}[K^2] - \mathrm{E}[K]^2 = np(1-p).$$

Characteristic function

$$\varphi_K(t) = \mathrm{E}[e^{itK}] = (1 - p + pe^{it})^n.$$

Characteristic functions are central to many proofs of limit theorems.

Cumulant-generating function

$$C_K(t) = \log M_K(t) = n \log(1 - p + pe^t).$$

Cumulants add under independent sums, which explains why binomial moments are especially easy to derive from Bernoulli increments.

Many balls and the multinomial histogram

Suppose M independent balls are released.

Let

$$N_k$$

be the number of balls settling in bin k, and define

$$\pi_k = P(K = k).$$

The complete count vector has a multinomial distribution:

$$(N_0, N_1, \dots, N_n) \sim \text{Multinomial}(M; \pi_0, \pi_1, \dots, \pi_n).$$

The expected bin count is

$$\mathrm{E}[N_k] = M\pi_k.$$

The variance is

$$\mathrm{Var}(N_k) = M\pi_k(1 - \pi_k).$$

For two different bins $j \neq k$,

$$\mathrm{Cov}(N_j, N_k) = -M\pi_j\pi_k.$$

The covariance is negative because a ball entering one bin cannot simultaneously enter another.

Define the empirical frequency

$$\hat{\pi}_k = \frac{N_k}{M}.$$

Then

$$\mathrm{E}[\hat{\pi}_k] = \pi_k$$

and

$$\mathrm{Var}(\hat{\pi}_k) = \frac{\pi_k(1-\pi_k)}{M}.$$

The standard error of an observed bin fraction is

$$\mathrm{SE}(\hat{\pi}_k) = \sqrt{\frac{\pi_k(1-\pi_k)}{M}}.$$

The histogram becomes less noisy as M increases because standard errors decrease as $1/\sqrt{M}$.

The law of large numbers

The law of large numbers describes convergence of averages.

For one ball, let $X_1, \ldots, X_n$ be row decisions. As the number of rows increases,

$$\frac{1}{n}\sum_{i=1}^{n} X_i \longrightarrow p$$

in probability, and under the strong law, almost surely.

Equivalently,

$$\frac{K}{n} \longrightarrow p.$$

This statement concerns one long sequence of row decisions.

A second law-of-large-numbers interpretation concerns many balls. For a fixed board,

$$\hat{\pi}_k = \frac{N_k}{M} \longrightarrow \pi_k$$

as the number of balls M increases.

These are different limiting operations:

- increasing n changes the number of peg rows and the shape of the single-ball distribution;
- increasing M improves the empirical estimate of the fixed distribution.

The simulator mainly demonstrates the second effect during a run: as more balls settle, the observed histogram approaches the theoretical binomial probabilities for the configured board.

The central limit theorem

General idea

The central limit theorem explains why properly standardised sums of many small independent contributions often approach a normal distribution.

For independent identically distributed variables X_i with finite mean μ_X and finite, non-zero variance σ_X^2,

$$Z_n = \frac{\sum_{i=1}^{n} X_i - n\mu_X}{\sigma_X \sqrt{n}}$$

converges in distribution to the standard normal law:

$$Z_n \xrightarrow{d} N(0,1).$$

Applied to Bernoulli trials

For Bernoulli variables,

$$\mu_X = p, \qquad \sigma_X^2 = p(1-p).$$

Therefore,

$$\frac{K - np}{\sqrt{np(1-p)}} \xrightarrow{d} N(0,1).$$

This is the central-limit-theorem interpretation of the Galton board.

What the theorem does and does not say

The theorem does say:

- a standardised binomial distribution approaches a normal distribution as n grows;
- many different microscopic distributions can produce approximately normal standardised sums;
- the approximation concerns distributions, not identical individual trajectories.

The theorem does not say:

- every random process is normal;
- every finite board produces an exact normal curve;
- independence is irrelevant;
- infinite variance causes no difficulty;
- a real mechanical board must be unbiased;

- a small sample histogram must look smooth;
- the normal approximation is equally accurate in the centre and tails.

Two sources of approximation error

A displayed histogram differs from a smooth normal curve for two independent reasons:

1. **Distributional approximation error** - the binomial distribution is discrete and only approximately normal for finite n.
2. **Sampling error** - only finitely many balls have been observed.

A board can have many balls but too few rows, producing a very accurate estimate of a visibly discrete binomial distribution. It can also have many rows but too few balls, producing a noisy histogram of a nearly normal underlying distribution.

The de Moivre-Laplace normal approximation

For

$$K \sim \text{Binomial}(n, p),$$

define

$$\mu = np, \qquad \sigma^2 = np(1-p).$$

The matching normal density is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right].$$

A local approximation to the binomial probability mass is

$$P(K = k) \approx \frac{1}{\sqrt{2\pi np(1-p)}} \exp\left[-\frac{(k-np)^2}{2np(1-p)}\right].$$

The approximation is best near the centre and generally improves when both

$$np$$

and

$$n(1-p)$$

are sufficiently large.

Common classroom guidelines require both quantities to be at least 5 or at least 10. These are practical heuristics, not mathematical boundaries.

Continuity correction

The binomial distribution is discrete, whereas the normal distribution is continuous.

To approximate

$$P(K = k),$$

use

$$P(k - 0.5 < Y < k + 0.5),$$

where

$$Y \sim N(np, np(1-p)).$$

For an interval,

$$P(a \leq K \leq b) \approx P(a - 0.5 < Y < b + 0.5).$$

Using the standard normal cumulative distribution function Φ,

$$P(a \leq K \leq b) \approx \Phi\left(\frac{b + 0.5 - np}{\sqrt{np(1-p)}}\right) - \Phi\left(\frac{a - 0.5 - np}{\sqrt{np(1-p)}}\right).$$

Continuity correction can substantially improve finite-n approximations.

Accuracy and the Berry-Esseen viewpoint

The central limit theorem is asymptotic. A Berry-Esseen bound quantifies convergence.

For independent identically distributed variables with finite third absolute central moment,

$$\sup_x |P(Z_n \leq x) - \Phi(x)| \leq \frac{C\rho}{\sigma_X^3 \sqrt{n}},$$

where

$$\rho = \mathrm{E}[|X - \mu_X|^3]$$

and C is a universal constant.

The key qualitative result is the rate

$$O(n^{-1/2}).$$

For Bernoulli trials, convergence can be slower when p is close to 0 or 1 because the distribution is strongly skewed and one side has limited expected counts.

The normal approximation is also less reliable in extreme tails. Exact binomial probabilities should be used when tail accuracy matters.

Empirical statistics and goodness of fit

The simulator reports observed and expected mean and standard deviation.

Observed mean bin

With bin counts N_k and total settled balls M,

$$\overline{K} = \frac{1}{M} \sum_{k=0}^{n} k N_k.$$

Observed population variance

The simulator treats the settled balls as the complete simulated population and uses

$$s_M^2 = \frac{1}{M} \sum_{k=0}^{n} N_k (k - \overline{K})^2.$$

The displayed standard deviation is

$$s_M = \sqrt{s_M^2}.$$

For inference from a sample to an external population, an unbiased sample-variance estimator would instead use $M - 1$:

$$s^2 = \frac{1}{M-1} \sum_{j=1}^{M} (K_j - \overline{K})^2.$$

Expected mean and variance

For a fixed p,

$$\mu = np, \qquad \sigma^2 = np(1-p).$$

The script can change p during a run. It therefore accumulates each settled ball's complete theoretical probability vector. The resulting displayed expectation is a mixture of the binomial distributions that were active when the balls were generated.

Pearson chi-square statistic

A formal goodness-of-fit measure is

$$\chi^2 = \sum_{k=0}^{n} \frac{(N_k - E_k)^2}{E_k},$$

where

$$E_k = M\pi_k.$$

Bins with very small expected counts should normally be combined before using the standard chi-square approximation.

Root-mean-square error

A simple descriptive discrepancy is

$$\mathrm{RMSE} = \sqrt{\frac{1}{n+1}\sum_{k=0}^{n}(\hat{\pi}_k - \pi_k)^2}.$$

Total variation distance

$$d_{\mathrm{TV}} = \frac{1}{2}\sum_{k=0}^{n}|\hat{\pi}_k - \pi_k|.$$

This is the maximum discrepancy between observed and theoretical probabilities over all events defined on the bins.

Kullback-Leibler divergence

When all required probabilities are positive,

$$D_{\mathrm{KL}}(\hat{\pi}\|\pi) = \sum_{k=0}^{n}\hat{\pi}_k \log\frac{\hat{\pi}_k}{\pi_k}.$$

Care is required when observed or expected probabilities are zero. Smoothing or omission conventions must be stated explicitly.

Sampling fluctuations are expected

A histogram is not expected to match theory exactly. For a bin with probability π_k, the approximate 95% fluctuation scale for its count is

$$M\pi_k \pm 1.96\sqrt{M\pi_k(1-\pi_k)}.$$

This interval is only a rough marginal guide and does not account for simultaneous comparison of all bins.

Biased and generalised Galton boards

Constant bias

If

$$p \neq \frac{1}{2},$$

the board is biased.

Then

$$K \sim \text{Binomial}(n, p),$$

with

$$\mu = np$$

and

$$\sigma^2 = np(1-p).$$

The distribution shifts toward the favoured side and becomes skewed for finite n.

The simulator exposes p as the **Right probability** control.

Row-dependent probabilities

If row i has probability p_i, then

$$K = \sum_{i=1}^{n} X_i, \qquad X_i \sim \text{Bernoulli}(p_i),$$

but the variables are no longer identically distributed.

The result is a **Poisson-binomial distribution**:

$$P(K = k) = [z^k] \prod_{i=1}^{n} (1 - p_i + p_i z),$$

where $[z^k]$ denotes the coefficient of z^k.

Its mean and variance are

$$\mathrm{E}[K] = \sum_{i=1}^{n} p_i,$$

$$\mathrm{Var}(K) = \sum_{i=1}^{n} p_i(1 - p_i),$$

assuming independence.

Dependent decisions

If later decisions depend on earlier ones, the sum need not be binomial.

Dependence may arise physically from:

- persistent spin;
- horizontal momentum;
- systematic peg misalignment;
- channel effects;
- repeated wall contacts.

A dependent board may be represented by a Markov chain or another stochastic process.

Multiplicative boards

An additive update has the form

$$X_{m+1} = X_m + \eta_m.$$

Repeated additive increments are associated with Gaussian limits under central-limit conditions.

A multiplicative update has the form

$$X_{m+1} = X_m A_m.$$

Taking logarithms gives

$$\log X_{m+1} = \log X_m + \log A_m.$$

If the logarithmic increments satisfy a central limit theorem, X_m can approach a lognormal distribution.

Contractive or memory-weighted updates

A recursion such as

$$X_{m+1} = (1 - \lambda)X_m + \eta_m, \qquad 0 < \lambda < 1,$$

reduces the influence of older increments. Under suitable assumptions, it can produce a stationary distribution with bounded variance.

This type of contraction is mathematically related to Galton's historical attempt to combine random deviation with reversion toward a central value.

Hilbert-Galton board

The Hilbert-Galton board is a continuous-time Markov-chain generalisation in which:

- balls arrive in bins at specified rates;
- a shift operation moves every retained bin one place;
- a new bin is inserted;
- stationary distributions and spectral properties can be analysed.

It is conceptually related to the classical board but is not simulated by this program.

Infinite mechanical boards and Lorentz gases

An idealised infinite board under a constant external force is related to a driven periodic Lorentz gas. This system involves:

- continuous position and velocity;
- elastic or modified collisions;
- invariant or non-invariant measures;
- hyperbolic billiard dynamics;
- recurrence;
- anomalous scaling.

It is mathematically distinct from the finite independent-Bernoulli board.

Physical mechanics of a real board

The current program is not a rigid-body dynamics solver, but the physical equations provide useful context.

Free flight under gravity

Between impacts, a point-mass model satisfies

$$\ddot{x} = 0, \qquad \ddot{y} = -g.$$

For initial state $(x_0, y_0, v_{x0}, v_{y0})$,

$$x(t) = x_0 + v_{x0}t,$$

$$y(t) = y_0 + v_{y0}t - \frac{1}{2}gt^2.$$

Collision with a fixed peg

Let n be the unit normal at contact and t a tangent.

Decompose the incoming velocity:

$$\mathrm{v}^- = v_n^- \mathrm{n} + v_t^- \mathrm{t}.$$

For a frictionless collision with coefficient of restitution e,

$$v_n^+ = -e v_n^-,$$

$$v_t^+ = v_t^-.$$

The vector form is

$$\mathrm{v}^+ = \mathrm{v}^- - (1+e)(\mathrm{v}^- \cdot \mathrm{n})\mathrm{n}.$$

The coefficient satisfies approximately

$$0 \le e \le 1.$$

- $e = 1$ is perfectly elastic in the normal direction.
- $e = 0$ removes all outgoing normal speed in the idealised law.
- Real materials may have speed-dependent restitution and rotational energy exchange.

Energy

Between impacts,

$$E = \frac{1}{2} m |\mathrm{v}|^2 + mgy$$

is conserved in an ideal gravitational model.

At an inelastic impact, translational kinetic energy is reduced. The normal kinetic-energy ratio is

$$\frac{T_n^+}{T_n^-} = e^2.$$

Why mechanics does not guarantee a binomial result

A mechanical board produces the ideal binomial law only approximately and only when its effective left/right decisions are:

- nearly symmetric;
- sufficiently independent;
- stable across rows and balls;

- not dominated by boundaries;
- not strongly correlated through spin or momentum.

The physical studies in the bibliography show that elasticity, peg radius, release conditions, and board dimensions can alter the observed distribution.

Probability model used by this simulator

The simulator intentionally gives priority to exact probability over detailed collision mechanics.

For each released ball:

1. Read the current right probability p.
2. For each of the n rows, generate one independent uniform pseudorandom number U_i.
3. Choose right when

$$U_i < p.$$

4. Choose left otherwise.
5. Count the number of right moves:

$$K = \sum_{i=1}^{n} X_i.$$

6. Assign the ball to bin K.
7. Construct smooth intermediate path nodes consistent with those decisions.
8. Animate the ball through those nodes.
9. When the ball settles, increment the observed count and add its theoretical probability vector to the accumulated expected counts.

This design guarantees that the final-bin law is binomial under a constant p.

Separation of model and animation

The following controls do not change the probability law:

- fall speed;
- bounce amplitude;
- ball size;
- impact intensity;
- marble gloss;
- board lighting;
- trails;
- glass perimeter;
- settled-ball-pile display.

The following control does change the probability law:

- right probability.

The release rate changes the rate at which samples are generated, not their distribution.

Worked example for the default 12-row board

The script uses

$$n = 12$$

by default.

For

$$p = \frac{1}{2},$$

the Pascal coefficients are

`1, 12, 66, 220, 495, 792, 924, 792, 495, 220, 66, 12, 1`

Their sum is

$$2^{12} = 4096.$$

The exact probabilities are

$$P(K = k) = \frac{\binom{12}{k}}{4096}.$$

Bin k	$\binom{12}{k}$	Probability	Percentage
0	1	0.000244	0.0244%
1	12	0.002930	0.2930%
2	66	0.016113	1.6113%
3	220	0.053711	5.3711%
4	495	0.120850	12.0850%
5	792	0.193359	19.3359%
6	924	0.225586	22.5586%
7	792	0.193359	19.3359%
8	495	0.120850	12.0850%
9	220	0.053711	5.3711%
10	66	0.016113	1.6113%
11	12	0.002930	0.2930%
12	1	0.000244	0.0244%

The theoretical moments are

$$\mu = np = 12\left(\frac{1}{2}\right) = 6,$$

$$\sigma^2 = np(1-p) = 12\left(\frac{1}{2}\right)\left(\frac{1}{2}\right) = 3,$$

$$\sigma = \sqrt{3} \approx 1.73205.$$

For $M = 10\,000$ balls, the expected central-bin count is

$$10\,000\frac{924}{4096} \approx 2255.86.$$

Its count standard deviation is

$$\sqrt{10\,000\left(\frac{924}{4096}\right)\left(1-\frac{924}{4096}\right)} \approx 41.80.$$

Therefore, a central-bin count several tens away from 2256 is normal sampling variation and is not evidence of an error by itself.

Program features

The current simulator includes:

- maximised startup on Windows;
- responsive board geometry;
- exact Bernoulli path generation;
- configurable right probability;
- configurable automatic release rate;
- single-ball and 100-ball release commands;
- configurable fall speed;
- configurable visual bounce amplitude;
- configurable ball size;
- configurable collision-effect intensity;
- configurable marble gloss;
- configurable board lighting;
- gray and white ball palette for contrast;
- trails;
- expanding impact rings;
- short spark particles;
- subtle protective-glass perimeter;
- representative packed balls in receiving bins;
- observed bin counts;
- exact accumulated theoretical curve;
- observed and expected mean;
- observed and expected standard deviation;
- Pascal coefficients and expected percentages;
- formula annotations;

- live-ball count;
- smoothed frame-rate display;
- help overlay;
- safety limits on live and total balls;
- standard-library-only runtime.

Software architecture

The program uses Python's standard library:

- `tkinter` and `tkinter.ttk` for the graphical interface;
- `math` for formulas and geometry;
- `random` for Bernoulli trials and visual variation;
- `time.perf_counter` for animation timing;
- `dataclasses` for structured state;
- `collections.deque` for bounded trails;
- type hints for readability and maintenance.

No numerical array package is needed because the model is small and the per-frame data are bounded.

Main classes

```
ImpactEffect
Spark
Ball
Layout
GaltonBoardApp
```

Main logical layers

1. **Configuration layer** - Tk variables store user-controlled values.
2. **Probability layer** - exact Bernoulli paths and binomial probabilities.
3. **State layer** - balls, effects, counts, expected counts, and timing.
4. **Animation layer** - smooth interpolation along precomputed path nodes.
5. **Rendering layer** - complete Canvas redraw in a fixed layer order.
6. **Interface layer** - controls, statistics, keyboard shortcuts, and help.

Data structures

`ImpactEffect`

Stores:

- impact position;
- ball colour;
- effect strength;
- current age;

- duration.

The effect expands and fades until its lifetime expires.

`Spark`

Stores:

- position;
- velocity;
- lifetime;
- size;
- age.

Sparks are decorative and do not interact with balls or pegs.

`Ball`

Stores:

- the complete precomputed path;
- path indices corresponding to peg impacts;
- final bin;
- right probability active at release;
- radius;
- gray/white shade;
- base animation speed;
- current path segment;
- progress through the segment;
- current position;
- settled flag;
- reflection phase;
- bounded trail history.

The final bin is determined before animation begins.

`Layout`

Stores the responsive geometry for one frame:

- Canvas dimensions;
- board bounds;
- inner-panel bounds;
- peg spacing;
- peg radius;
- ball radius;
- funnel position;
- bin bounds;
- bin width;
- number of bins.

`Layout` is immutable. The same instance is used during update and rendering for a frame.

`GaltonBoardApp`

Owns:

- Tk widgets;
- simulation parameters;
- object lists;
- observed counts;
- expected counts;
- event bindings;
- animation timing;
- probability calculations;
- drawing routines.

Simulation algorithm

Ball creation

The method `_spawn_ball()`:

1. checks `MAX_LIVE_BALLS`;
2. checks `MAX_TOTAL_BALLS`;
3. reads the current p;
4. calls `_generate_path()`;
5. assigns size, shade, speed, and reflection phase;
6. appends the new ball to the active list;
7. increments the release counter.

Path generation

The method `_generate_path()` performs the mathematical experiment.

Conceptual pseudocode:

```
rights = 0
path = [release_point]

for each row:
    add approach-to-peg node

    if uniform_random_number < p:
        direction = right
        rights += 1
    else:
        direction = left

    add visual side-deflection node
    add next-lattice-position node

bin_index = rights
add nodes leading to centre of receiving bin
return path, impact_indices, bin_index
```

The side-deflection node depends on bounce amplitude. The next lattice position always shifts by exactly one half peg spacing left or right, preserving the triangular lattice.

Motion interpolation

The method `_advance_ball()` interpolates between path nodes.

Let $t \in [0, 1]$ be segment progress. The simulator uses cubic smoothstep:

$$s(t) = 3t^2 - 2t^3.$$

This satisfies

$$s(0) = 0, \qquad s(1) = 1,$$

$$s'(0) = 0, \qquad s'(1) = 0.$$

The displayed position is

$$\mathrm{x}(t) = \mathrm{x}_0 + s(t)(\mathrm{x}_1 - \mathrm{x}_0).$$

Smoothstep prevents abrupt velocity discontinuities at the visual nodes.

The algorithm can consume more than one path segment in one frame. This prevents motion from slowing incorrectly when frame time varies.

Settling

When a ball reaches its final path node:

- the appropriate integer bin count is incremented;
- total settled balls is incremented;
- the ball's binomial probability vector is added to expected counts;
- the ball is removed from the active list.

Animation and rendering

The application uses a single Tk event loop.

The `_tick()` method:

1. reads the monotonic clock;
2. computes elapsed time;
3. caps elapsed time at `0.033` seconds;
4. updates simulation state;
5. redraws the Canvas;
6. schedules the next frame with `root.after()`.

The target rate is

$$60 \text{ frames per second.}$$

The nominal scheduling interval is approximately

$$\frac{1000}{60} \approx 16.67 \text{ ms.}$$

Actual rate depends on hardware, window size, active balls, operating-system scheduling, and Tk rendering performance.

Frame-time cap

The cap

$$\Delta t \leq 0.033 \text{ s}$$

prevents a delayed frame from creating a very large motion jump after:

- window dragging;
- temporary system load;
- debugger pauses;
- returning from a blocked application state.

Layer order

The Canvas is drawn back-to-front:

1. outer background;
2. board shadow;
3. frame;
4. inner board surface;
5. bins;
6. pegs;
7. scientific annotations;
8. impacts;
9. sparks;
10. moving balls;
11. glass perimeter;
12. title plate;
13. compact status;
14. optional help overlay.

This order ensures that balls appear above pegs and annotations while the help panel appears above the complete scene.

Observed and theoretical distributions

Observed counts

`bin_counts[k]` is the exact number of settled balls in bin k.

Expected counts for fixed probability

If all balls use the same p and M balls have settled,

$$E_k = M\binom{n}{k}p^k(1-p)^{n-k}.$$

Expected counts when probability changes

Suppose ball j is released with probability p_j. Its theoretical contribution to bin k is

$$\pi_{j,k} = \binom{n}{k}p_j^k(1-p_j)^{n-k}.$$

After M balls,

$$E_k = \sum_{j=1}^{M}\pi_{j,k}.$$

The script implements this accumulated expectation. It is more rigorous than retroactively applying the current slider value to all previously released balls.

The aggregate distribution is a mixture:

$$\overline{\pi}_k = \frac{1}{M}\sum_{j=1}^{M}\pi_{j,k}.$$

Interface controls

Control	Meaning	Changes probability?
Release rate	Automatic balls per second	No
Fall speed	Animation speed multiplier	No
Right probability	Bernoulli parameter p	**Yes**
Bounce amplitude	Visual side deflection	No
Ball size	Radius of newly released balls	No
Impact intensity	Collision-ring and spark strength	No
Marble gloss	Ball highlight intensity	No
Board lighting	Frame and metal highlight intensity	No
Trails	Show bounded motion history	No
Impacts	Enable rings and sparks	No
Glass	Show subtle perimeter	No

Control	Meaning	Changes probability?
Marble piles	Show representative settled balls	No
Theory	Show accumulated theoretical curve	No

Default values

Parameter	Default
Rows	12
Target FPS	60
Maximum live balls	420
Maximum total releases	50,000
Release rate	14 balls/s
Fall speed	0.72
Right probability	0.50
Bounce amplitude	0.90
Ball scale	1.00
Impact intensity	0.82
Marble gloss	0.88
Board lighting	0.84

Keyboard controls

Key	Operation
`Space`	Pause/resume
`R`	Reset
`N`	Release one ball
`B`	Queue 100 balls
`Up Arrow`	Increase release rate
`Down Arrow`	Decrease release rate
`H`	Toggle help overlay
`Esc`	Close program

Reset clears:

- active balls;
- impact effects;
- spark particles;
- observed bin counts;
- expected bin counts;
- release and settled counters;
- release timing accumulator;
- burst queue.

Reset retains:

- parameter settings;
- check-box states;
- window size;
- maximised state.

Installation

1. Install Python

Install Python 3.10 or newer from the official Python distribution for Windows.

During installation:

- enable **Add Python to PATH** if desired;
- include **Tcl/Tk and IDLE**;
- include `pip`;
- include the Python launcher `py` if available.

Confirm installation:

```
python --version
```

or

```
py --version
```

2. Verify Tkinter

```
python -m tkinter
```

A small Tk test window should open.

If the command fails, repair or reinstall Python and ensure the Tcl/Tk component is installed.

3. Clone or download the repository

Using Git:

```
git clone <repository-url>
cd <repository-directory>
```

Alternatively, download the repository ZIP, extract it, and open Command Prompt in the extracted directory.

4. No runtime pip dependencies

The simulator uses the Python standard library only.

Do not install NumPy, SciPy, Matplotlib, Pygame, or Pillow unless you separately modify the program to use them.

Running the simulator

From the repository directory:

```
python script.py
```

If the script has been renamed:

```
python script.py
```

Run through the Python launcher

```
py -3.10 script.py
```

Run without a console window

On Windows, `pythonw.exe` starts a graphical script without a Command Prompt console:

```
pythonw script.py
```

For debugging, prefer `python.exe` so tracebacks remain visible.

Virtual environment setup

A virtual environment is not required to run the standard-library-only script. It is recommended for reproducible packaging tools.

Windows Command Prompt

```
py -3.10 -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip
```

The prompt should show `(.venv)`.

Windows PowerShell

```
py -3.10 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Git Bash

```
py -3.10 -m venv .venv
source .venv/Scripts/activate
python -m pip install --upgrade pip
```

Verify the active interpreter

```
where python
python -c "import sys; print(sys.executable)"
```

The path should point into `.venv`.

Deactivate

```
deactivate
```

Remove the environment

Delete the `.venv` directory. It can be recreated from the commands above.

Packaging with PyInstaller

What PyInstaller does

PyInstaller is often informally called a Python compiler. More precisely, it analyses imports and bundles:

- the Python interpreter;
- the bytecode or source-derived application modules;
- the standard-library modules used by the program;
- Tk/Tcl runtime files;
- the executable bootloader;
- optional resources.

It produces an application that can run on a compatible Windows machine without a separate Python installation.

PyInstaller is not a general cross-compiler. Build the Windows executable on Windows, preferably on a system similar to the target system.

Install PyInstaller inside the virtual environment

```
python -m pip install pyinstaller
```

Verify:

```
python -m PyInstaller --version
```

Recommended one-directory build

A one-directory package starts faster and is easier to inspect:

```
python -m PyInstaller ^
  --noconfirm ^
  --clean ^
  --windowed ^
  --onedir ^
  --name GaltonBoardSimulator ^
  script.py
```

Output:

```
dist\GaltonBoardSimulator\GaltonBoardSimulator.exe
```

Distribute the entire `GaltonBoardSimulator` directory.

Single-file build

```
python -m PyInstaller ^
  --noconfirm ^
  --clean ^
  --windowed ^
  --onefile ^
  --name GaltonBoardSimulator ^
  script.py
```

Output:

```
dist\GaltonBoardSimulator.exe
```

A one-file executable normally starts more slowly because its embedded files are extracted to a temporary directory at launch.

Build with an icon

```
python -m PyInstaller ^
  --noconfirm ^
  --clean ^
  --windowed ^
  --onefile ^
  --name GaltonBoardSimulator ^
  --icon assets\galton.ico ^
  script.py
```

The icon must be a Windows `.ico` file.

Clean rebuild

```
rmdir /s /q build
rmdir /s /q dist
del GaltonBoardSimulator.spec

python -m PyInstaller ^
  --noconfirm ^
  --clean ^
  --windowed ^
  --onefile ^
  --name GaltonBoardSimulator ^
  script.py
```

Build without activating the environment

```
.venv\Scripts\python.exe -m PyInstaller ^
  --noconfirm ^
  --clean ^
  --windowed ^
  --onefile ^
  --name GaltonBoardSimulator ^
  script.py
```

Console build for debugging

Temporarily omit `--windowed`:

```
python -m PyInstaller ^
  --noconfirm ^
  --clean ^
  --onefile ^
  --name GaltonBoardSimulatorDebug ^
  script.py
```

The console displays exceptions and diagnostic output.

Customising the simulator

Change the number of rows

In the class definition:

```
ROWS = 12
```

Change to another positive integer:

```
ROWS = 16
```

Consequences:

- bins become `ROWS + 1`;
- Pascal coefficients change;
- the theoretical distribution changes;
- geometry is recalculated;
- more rows increase path length and animation time;
- board annotations may require layout adjustments for very large values.

The current interface is designed primarily for moderate row counts.

Change the default probability

```
self.right_probability = tk.DoubleVar(value=0.50)
```

Example:

```
self.right_probability = tk.DoubleVar(value=0.60)
```

Expand the probability-slider range

Locate the `Right probability` slider definition and change its lower and upper bounds.

The internal probability calculation clamps values to $[0, 1]$.

Change the random seed

For reproducible demonstrations, add a seed before any balls are generated:

```
random.seed(12345)
```

A suitable location is near the start of `__init__`.

Do not set the seed repeatedly for each ball, because doing so can recreate identical sequences.

Change colours

The palette constants are near the top of the script.

Gray/white ball shades are defined in:

```
BALL_COLORS = [
    ...
]
```

Change safety limits

```
MAX_LIVE_BALLS = 420
MAX_TOTAL_BALLS = 50_000
```

Increasing these values may reduce frame rate and memory responsiveness.

Change target frame rate

```
TARGET_FPS = 60
```

Tk timing is not a hard real-time scheduler. Increasing this value does not guarantee a higher actual frame rate.

Add output export

Potential extensions include:

- CSV export of bin counts;
- JSON export of settings and counts;
- screenshot export;
- run metadata;
- deterministic seed recording;
- chi-square statistics;
- batch experiments without animation.

A rigorous export should record at least:

- row count;
- probability;
- total balls;
- random seed if fixed;
- observed counts;
- expected counts;
- timestamp;
- software version.

Verification experiments

Experiment 1 - Symmetry

Set:

$$p = 0.5.$$

Release several thousand balls.

Expected:

$$N_k \approx N_{n-k}.$$

Small differences are sampling variation.

Experiment 2 - Mean and standard deviation

For the default board:

$$n = 12, \qquad p = 0.5.$$

Expected:

$$\mu = 6,$$

$$\sigma = \sqrt{3} \approx 1.732.$$

The observed values should approach these as the number of settled balls increases.

Experiment 3 - Bias

Set:

$$p = 0.60.$$

Expected:

$$\mu = np = 7.2,$$

$$\sigma^2 = np(1-p) = 2.88,$$

$$\sigma \approx 1.697.$$

The histogram should shift toward larger bin numbers.

Experiment 4 - Extreme bins

For $n = 12$ and $p = 0.5$,

$$P(K = 0) = P(K = 12) = \frac{1}{4096}.$$

An edge-bin ball is therefore rare but valid. In 4096 balls, the expected count in each extreme bin is one, but observing zero, one, two, or more remains possible.

Experiment 5 - Visual controls do not alter statistics

Run two large experiments with the same seed and probability but different:

- fall speed;
- bounce amplitude;
- gloss;
- lighting;
- trails;
- impact intensity.

If the seed and sequence of random calls are controlled identically, the probability-model outcomes should match. In the current code, some decorative properties also use random numbers, so exact cross-configuration path reproduction requires separating statistical and aesthetic random-number generators as described under Reproducibility.

Experiment 6 - Change probability during a run

1. Release balls with $p = 0.4$.
2. Change to $p = 0.6$.
3. Release more balls.

The expected curve should represent the accumulated mixture rather than a single Binomial$(n, 0.6)$ law applied to all balls.

Performance and numerical considerations

Computational complexity

For each released ball:

- path generation is $O(n)$;
- probability-vector accumulation at settling is $O(n)$;
- path animation work is proportional to traversed segments;
- drawing work is proportional to active balls, effects, pegs, and displayed pile marbles.

With fixed $n = 12$, these costs are small.

Memory

Each active ball stores:

- approximately three path nodes per row plus receiving-bin nodes;
- a bounded trail deque;
- scalar animation state.

Settled balls are not retained individually. Only aggregate counts remain. This prevents memory from growing linearly with total settled balls.

Live-ball cap

```
MAX_LIVE_BALLS = 420
```

This limits simultaneous animation load.

Total-release cap

```
MAX_TOTAL_BALLS = 50_000
```

This prevents an indefinitely running session from accumulating unbounded counters and expected-value updates.

Floating-point precision

Probability calculations use Python double-precision floating point.

For the default board, errors are negligible.

For very large n, direct evaluation of

$$\binom{n}{k} p^k (1-p)^{n-k}$$

can underflow or lose relative precision in the tails. More robust approaches include:

- logarithmic probabilities using `math.lgamma`;
- recursive adjacent-probability calculation;
- specialised statistical libraries;
- normal or saddlepoint approximations where justified.

Random-number generator

Python's default `random` module uses the Mersenne Twister pseudorandom generator.

It is suitable for educational Monte Carlo simulation. It is not intended for cryptographic use.

Rendering cost

Tk Canvas is immediate-mode and CPU-rendered. Large maximised windows, many live balls, many trails, and many effects increase draw calls.

For a faster run:

- disable trails;
- disable impacts;
- disable marble piles;
- reduce release rate;
- reduce maximum live balls;
- use one-directory packaging instead of one-file if startup is the concern.

Reproducibility

Fixed seed

Add:

```
random.seed(12345)
```

before the first ball is generated.

Separate statistical and visual random generators

The most rigorous design uses two random-number streams:

```
self.model_rng = random.Random(12345)
self.visual_rng = random.Random(67890)
```

Use `model_rng` only for left/right decisions:

```
direction = 1 if self.model_rng.random() < probability_right else -1
```

Use `visual_rng` for:

- ball shade;
- base speed;
- reflection phase;
- spark angle;
- spark speed;
- spark size.

This separation ensures that changing visual effects does not change the statistical path sequence.

Record configuration

For a reproducible run, record:

- software commit;
- Python version;
- operating system;
- row count;
- initial and changed probabilities;
- seed or seeds;
- number of balls;
- parameter values;
- final bin counts.

Limitations

1. **Not a rigid-body solver.**
   The animation does not integrate gravity and contact impulses to determine left/right outcomes.

2. **Independent-row assumption.**
   Each row decision is independent in the mathematical model.
3. **Constant probability within a ball.**
   A ball reads p at release and uses that value for all its rows.
4. **Pseudorandomness.**
   The generator is deterministic given its state.
5. **Finite number of rows.**
   The displayed distribution is binomial, not exactly continuous normal.
6. **Finite number of balls.**
   The histogram contains sampling noise.
7. **Representative pile drawing.**
   When counts are large, the visible number of packed marbles is normalised to drawing capacity.
8. **No data export in the current version.**
9. **No formal statistical test in the interface.**
10. **Tkinter performance limits.**
    The program is designed for an educational desktop animation, not high-throughput simulation.
11. **No accessibility audio mode.**
12. **No automated localisation.**
13. **No physical parameter calibration.**
    Peg radius, restitution, friction, and gravity are not model parameters.

Glossary

Bernoulli distribution
A two-outcome distribution with success probability p.

Bernoulli trial
One binary random experiment.

Bias
A departure from $p = 1/2$ in the left/right model.

Binomial coefficient
The combinatorial count $\binom{n}{k}$.

Binomial distribution
The distribution of the number of successes in n independent Bernoulli trials with common success probability p.

Central limit theorem
A family of theorems describing convergence of standardised sums toward a normal distribution

under suitable conditions.

Continuity correction
A half-unit boundary adjustment used when approximating a discrete distribution by a continuous one.

Expected value
The probability-weighted average of a random variable.

Galton board
A peg-and-bin apparatus illustrating repeated random deflections and aggregate probability distributions.

Histogram
A graphical representation of counts or frequencies across categories or intervals.

Independence
A property under which joint probabilities factor into products of marginal probabilities.

Law of large numbers
A theorem describing convergence of empirical averages or frequencies toward expected values.

Mean
Another name for expected value in the theoretical context, or arithmetic average in an empirical context.

Monte Carlo simulation
Numerical experimentation based on repeated pseudorandom samples.

Normal distribution
A continuous bell-shaped distribution characterised by mean and variance.

Pascal's triangle
A triangular arrangement of binomial coefficients.

Probability mass function
A function assigning probabilities to the possible values of a discrete random variable.

Pseudorandom number generator
A deterministic algorithm producing sequences designed to behave like random samples for specified purposes.

Quincunx
A historical name for Galton's staggered peg apparatus.

Random variable
A numerical function of the outcome of a random experiment.

Random walk
A process formed by successive random increments.

Regression toward the mean
The tendency for extreme observations selected under imperfect correlation to be followed, on average, by less extreme observations.

Sample space
The set of all possible elementary outcomes.

Standard deviation
The square root of variance.

Variance
The expected squared deviation from the mean.

Bibliography

The following sources support the historical, mathematical, physical, and computational discussion.

1. **Galton, F.** (1877). "Typical Laws of Heredity." *Proceedings of the Royal Institution of Great Britain*, **8**, 282-301. The lecture was delivered at the Royal Institution on 9 February 1877.

2. **Galton, F.** (1877). "Typical Laws of Heredity," Parts I-III. *Nature*, **15**, 492-495, 512-514, and 532-533.

3. **Galton, F.** (1889). *Natural Inheritance*. London: Macmillan.

4. **Galton, F.** (1869). *Hereditary Genius: An Inquiry into Its Laws and Consequences*. London: Macmillan.

5. **Pearl, J., and Mackenzie, D.** (2018). *The Book of Why: The New Science of Cause and Effect*. New York: Basic Books. The chapter excerpt discussing Galton's quincunx, heredity, and regression toward the mean was supplied with this project.

6. **Kozlov, V. V., and Mitrofanova, M. Yu.** (2003). "Galton Board." *Regular and Chaotic Dynamics*, **8**(4), 431-439. DOI: 10.1070/RD2003v008n04ABEH000255.

7. **Chernov, N., and Dolgopyat, D.** (2009). "The Galton Board: Limit Theorems and Recurrence." *Journal of the American Mathematical Society*, **22**(3), 821-858. DOI: 10.1090/S0894-0347-08-00626-7.

8. **Ayyer, A., and Ramassamy, S.** (2018). "The Hilbert-Galton Board." *ALEA, Latin American Journal of Probability and Mathematical Statistics*, **15**, 755-774. DOI: 10.30757/ALEA.v15-28.

9. **Auricchio, G., Ghiotto, M., Gualandi, S., and Toscani, G.** (2025). "Generalized Galton's Boards Explain Social Phenomena via Statistical Physics." Manuscript supplied with this project.

10. **Goldenberg, D. P.** (2024). *Physical Principles in Biology, Chapter 2: Probability*. University of Utah course notes, draft dated 30 December 2024.

11. **Bernoulli, J.** (1713). *Ars Conjectandi*. Basel: Thurneysen.

12. **de Moivre, A.** (1733). "Approximatio ad Summam Terminorum Binomii $(a+b)^n$ in Seriem Expansi." A foundational normal approximation to binomial probabilities.

13. **Laplace, P.-S.** (1812). *Theorie analytique des probabilites*. Paris.

14. **Pascal, B.** (1665). *Traite du triangle arithmetique*. Published posthumously.

15. **Quetelet, A.** (1846). *Lettres a S.A.R. le Duc regnant de Saxe-Cobourg et Gotha, sur la theorie des probabilites, appliquee aux sciences morales et politiques*. Brussels.

16. **Feller, W.** (1968). *An Introduction to Probability Theory and Its Applications*, Volume I, 3rd ed. New York: Wiley.

17. **Billingsley, P.** (2012). *Probability and Measure*, anniversary ed. Hoboken: Wiley.

18. **Stigler, S. M.** (1986). *The History of Statistics: The Measurement of Uncertainty before 1900*. Cambridge, MA: Harvard University Press.

19. **Hald, A.** (1990). *A History of Probability and Statistics and Their Applications before 1750*. New York: Wiley.

20. **Pareschi, L., and Toscani, G.** (2013). *Interacting Multiagent Systems: Kinetic Equations and Monte Carlo Methods*. Oxford: Oxford University Press.

Software citation

When citing the simulator in a report, thesis, teaching resource, or publication, use a record similar to:

```
Galton Board Simulator. Python/Tkinter implementation of an exact
Bernoulli-path Galton board with binomial reference distribution,
scientific annotations, animated trajectories, and configurable
visualisation parameters. Version or commit: <commit hash>. Accessed:
<date>. Repository: <repository URL>.
```

A BibTeX template is:

```
@software{galton_board_simulator,
  title        = {Galton Board Simulator},
  author       = {{Repository contributors}},
  year         = {2026},
  version      = {<version or commit>},
  url          = {<repository URL>},
  note         = {Python/Tkinter simulator of Bernoulli trials and the binomial
↪  distribution}
}
```

Replace placeholder values before publication.

Final technical distinction

The central scientific distinction in this repository is:

- the **probability model** is an exact sequence of independent Bernoulli trials;
- the **visual path** is a smooth animation of the selected sequence;
- the **theoretical curve** is the accumulated exact binomial expectation;
- the **normal curve** is a large-row approximation, not the exact finite-board law;
- the **real mechanical board** is a dynamical system whose agreement with the ideal model depends on physical design and calibration.

That distinction makes the simulator useful both as a probability demonstration and as an example of careful mathematical modelling.